

Introduction to Selenium

Selenium is an open-source and a portable automated software-testing Framework for testing web applications.

It has capabilities to operate across different browsers and operating systems.

Selenium is not just a single tool but also a set of tools that helps testers to automate web-based applications more efficiently.

It is widely used for functional testing, regression testing, and performance testing.

Selenium supports multiple programming languages, including Java, C#, Python, and Ruby, making it accessible to a wide range of developers.

Safari, Internet Explorer, Mozilla Firefox, and Google Chrome are some of the browsers that are supported by Selenium.

***Core Components of the Selenium Suite**

1. **Selenium WebDriver:** A tool for automating browser interactions and for controlling web browsers programmatically.
2. **Selenium IDE:** A browser-based tool for recording and playing back user interactions with a web application.
3. **Selenium Grid:** A tool for running tests in parallel on multiple machines, allowing for faster test execution and improved resource utilization.

***What is the use of selenium web driver?**

-Selenium WebDriver is primarily used to **programmatically automate web browser interactions** to test web applications.

-It acts as a code-based remote control that allows developers and quality assurance (QA) engineers to simulate real human behaviour in a browser—such as clicking buttons, typing text, filling out forms, and navigating between pages—to verify that a website functions correctly.

In older version we were installing Web Driver externally or separately.

Modern Selenium (Selenium 4+) automates this process using [Selenium Manager](#), meaning you no longer need to manually download or configure driver.

Selenese commands

Selenese commands in **Selenium IDE** use specific prefixes to target web elements. Selenese scripts follow a three-column structure consisting of a **Command** (e.g., click, type), a **Target** (the locator), and a **Value** (data being sent).

*Create selenese commands:

Locators: by ID, name, class, tag name, XPath

1. Locating by ID

Target elements using their unique HTML `id` attribute

Syntax: id=elementId

- **Example HTML:** <input id="username_field" type="text">
- **Selenese Format:**
 - **Command:** type
 - **Target:** id=username_field
 - **Value:** JohnDoe

2. Locating by Name

Target elements using their HTML `name` attribute, which is common in forms.

Syntax: name=elementName

- **Example HTML:** <input name="email_address" type="email">
- **Selenese Format:**
 - **Command:** type
 - **Target:** name=email_address
 - **Value:** john@example.com

3. Locating by Class Name

Target elements using their CSS styling class. In Selenese / Selenium IDE, class-based matching is natively written using the CSS prefix.

- **Syntax:** css=.className
- **Example HTML:** <button class="submit-btn-large">Submit</button>
- **Selenese Format:**
 - **Command:** click
 - **Target:** css=.submit-btn-large
 - **Value:** (leave blank)

4. Locating by Tag Name

Target elements based on their specific HTML element type. This is also written via CSS selectors within Selenese.

- **Syntax:** `css=tagName`
- **Example HTML:** `<th>Header Text</th>`
- **Selenese Format:**
 - **Command:** `assertText`
 - **Target:** `css=th`
 - **Value:** `Header Text`

5. Locating by XPath

Navigate through the XML/HTML document structure. This is essential for structural queries or targeting dynamic components.

- **Syntax:** `xpath=//expression`
- **Example HTML:** `<div class="container"><button>Save</button></div>`
- **Selenese Format:**
 - **Command:** `click`
 - **Target:** `xpath=//div[@class='container']/button`
 - **Value:** *(leave blank)*

UI Interactions in Automation Testing

1. Text Box Interaction

A text box is used to enter user input such as username, password, email, or search text.

Common Actions:

- Enter text
- Clear existing text
- Append text
- Verify entered value

Example:

Username: admin

Password: Admin@123

2. Radio Button Selection

Radio buttons allow users to select only one option from a group.

Common Actions:

- Select a radio button
- Verify whether it is selected
- Verify default selected option

Example: Gender:

☒ Male

☐ Female

☐ Other

3. Check Box Selection

Checkboxes allow users to select one or more options.

Common Actions:

- Check a checkbox
- Uncheck a checkbox
- Verify checked state
- Select multiple checkboxes

Example:

☒ Java

☒ Python

☐ C#

4. Drop-down Item Selection

A drop-down list provides multiple options from which the user selects one.

Selection Methods:

- Select by visible text
- Select by value
- Select by index

Example: Country

▼ **India**

5. Keyboard Actions

Automation tools can simulate keyboard operations.

Common Keyboard Actions:

- Enter
- Tab
- Escape
- Backspace
- Delete
- Ctrl + A
- Ctrl + C
- Ctrl + V
- Ctrl + X
- Arrow Keys
- Shift
- Function Keys

Use Cases

- Submit forms
- Navigate between fields
- Copy and paste
- Shortcut testing

6. Mouse Actions

Mouse actions simulate user interaction with UI elements.

Common Mouse Actions

- Click
- Double Click
- Right Click
- Hover (Mouse Over)
- Drag and Drop
- Click and Hold
- Move to Element

Use Cases

- Open menus

- Context menu testing
- Drag files
- Resize elements

Lab Practical's

- Download and configure Selenium
- Create a test suite
- Add commands and interactions

Step to complete Java Selenium Setup

1. Download and install JDK
2. Download and install Eclipse
3. Set environmental variable path for jdk
4. Verify environmental variable are set : `java -version`
5. Create new Maven Project in eclipse
6. Add selenium dependency
7. Set the driver path and launch URL (in latest version we can use it by directly creating instance of driver class)

Step3: Set environmental variable path for jdk => c drive => programme file => open java folder => open jdk folder and copy it url path (folder path) and also copy bin folder path

C:\Program Files\Java\jdk-21.0.10

C:\Program Files\Java\jdk-21.0.10\bin

Open system environmental variable option => click on environmental variable => in system variable section select path option and add new path and write

Variable name : JAVA_HOME

Variable Value: C:\Program Files\Java\jdk-21.0.10

Again select path variable => click on edit option => click on new option => and paste path
C:\Program Files\Java\jdk-17\bin

Step5 : Create new Maven Project in eclipse => open eclipse select file => select new Option => select maven project => select create a simple project => click on next

In Maven, `groupId` and `artifactId` serve as unique coordinates that identify your project across the entire Maven ecosystem.

What is Group ID (groupId)?

The `groupId` specifies the **organization, company, or group** that created the project.

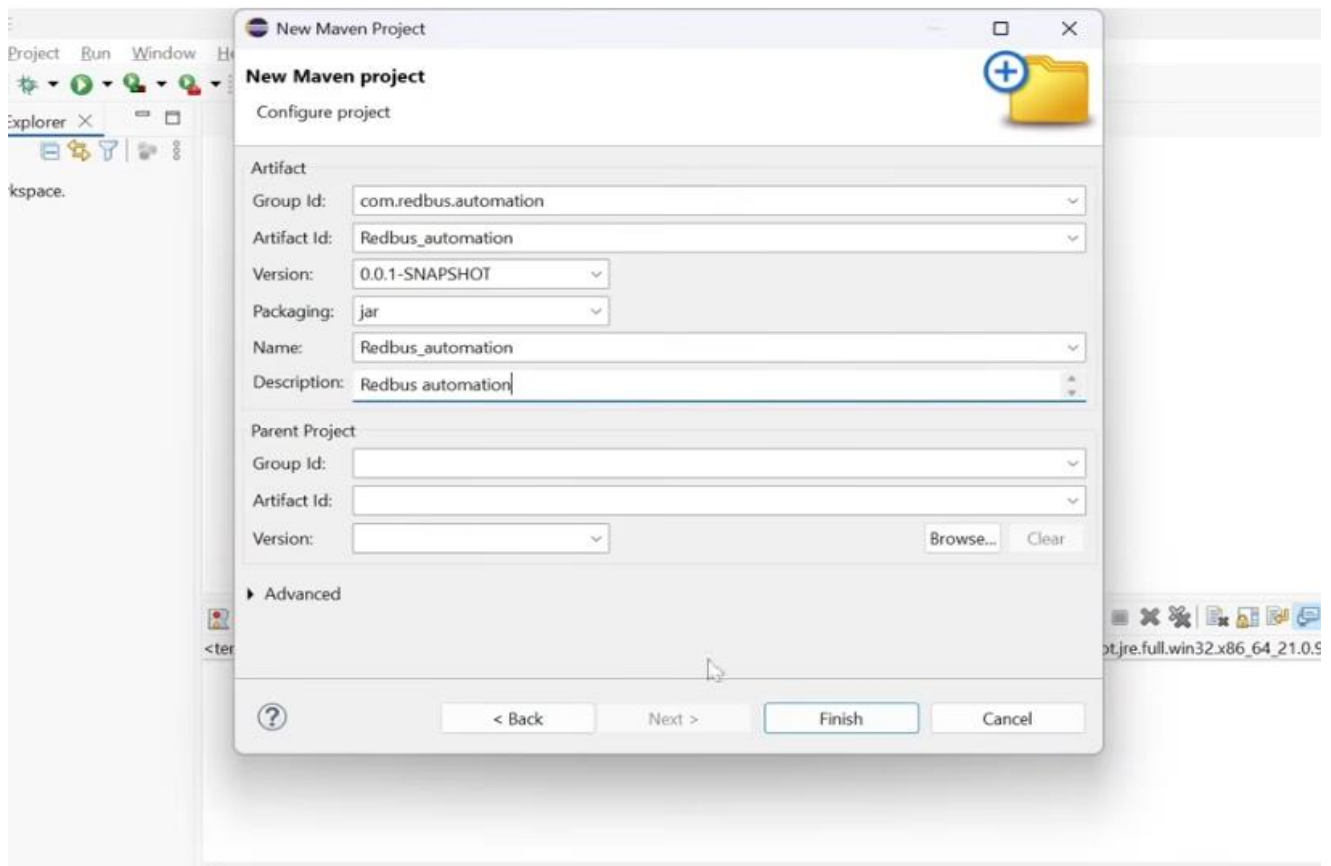
- Naming **Convention**: It must follow Java's package naming rules. You generally start with a reversed domain name that you own or control.

- Examples: `com.google`, `org.apache.maven`, Or `com.yourcompany.department`.

What is Artifact ID (artifactId)?

The `artifactId` is the **unique name of the project or module** itself.

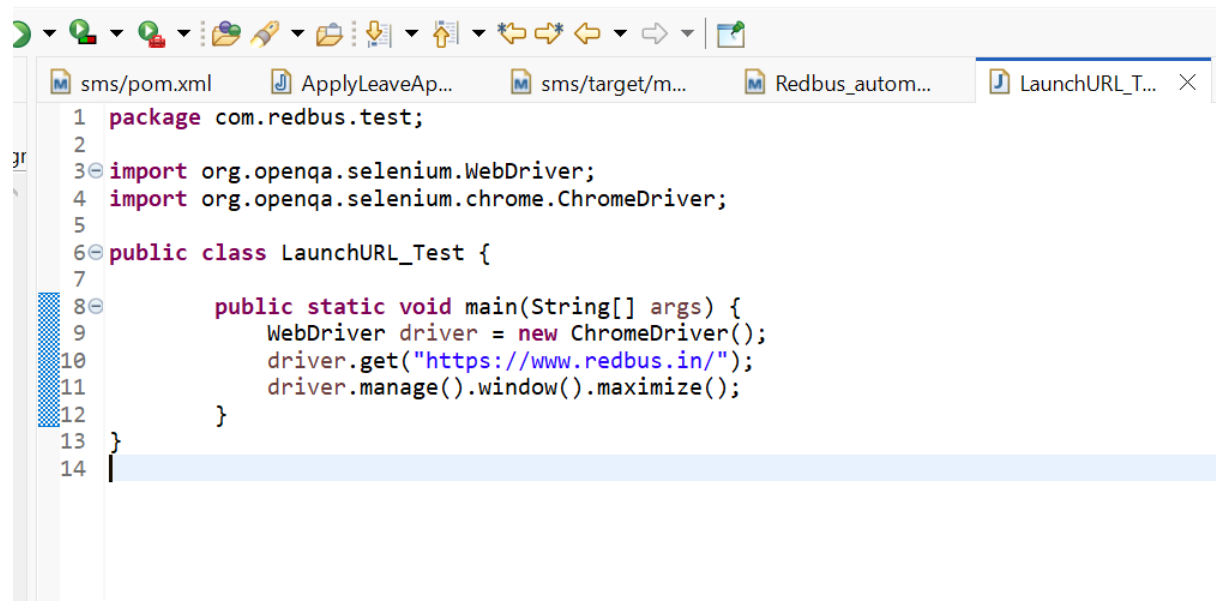
- **Naming Convention:** It represents the final name of the compiled file (like a JAR or WAR file) without the version number. It should use lowercase letters, digits, and hyphens.
- **Examples:** `user-service`, `payment-gateway`, Or `guava`.



6. Add selenium dependency => go to browser url and search “mvn repository “ => it will open mvn dashboard and search => selenium java => select selenium java dependency and open it => click on version => click on latest version => when you click on latest version it will open it , we have to scroll down and copy maven code => go to project and open pom.xml file and paste code inside <dependencies> <dependencies> tag as shown below

select `src\test\java` => right click on it => select New => create package => inside name = `com.redbus.test` => click on finish

right click on new create package=> select new => select class => give class name as LaunchURL_Test => click on finish



```
1 package com.redbus.test;
2
3 import org.openqa.selenium.WebDriver;
4 import org.openqa.selenium.chrome.ChromeDriver;
5
6 public class LaunchURL_Test {
7
8     public static void main(String[] args) {
9         WebDriver driver = new ChromeDriver();
10        driver.get("https://www.redbus.in/");
11        driver.manage().window().maximize();
12    }
13 }
14
```